

Crosslink: Real-Time Multiplayer Tic-Tac-Toe

Course: CS5700 Computer Networks

Project choice: Project 2 — Multiplayer Tic-Tac-Toe

Team members: Johnny He and Yiwen Hu

Source code: github.com/go-johnnyhe/Crosslink

Date: August 2026

1. Introduction

Crosslink is a browser-based multiplayer Tic-Tac-Toe application. It was built for Project 2 in the CS5700 final project specification. The game allows two people to connect from separate browser sessions, enter the same room, and play in real time. It also provides matchmaking, text chat, score tracking, connection activity, and rematches.

The main challenge was not the Tic-Tac-Toe rules. The main challenge was keeping two users synchronized over a network. If each browser managed its own board, the players could see different turns or results. Crosslink solves this problem by keeping one official copy of each room on the server. A browser sends an action to the server, the server checks it, and both browsers receive the updated state.

The system uses Next.js, React, TypeScript, HTTP route handlers, and Server-Sent Events (SSE). The design is intentionally small enough for a class project, but it demonstrates important networking ideas such as client-server communication, shared state, server-side validation, real-time delivery, and reconnection.

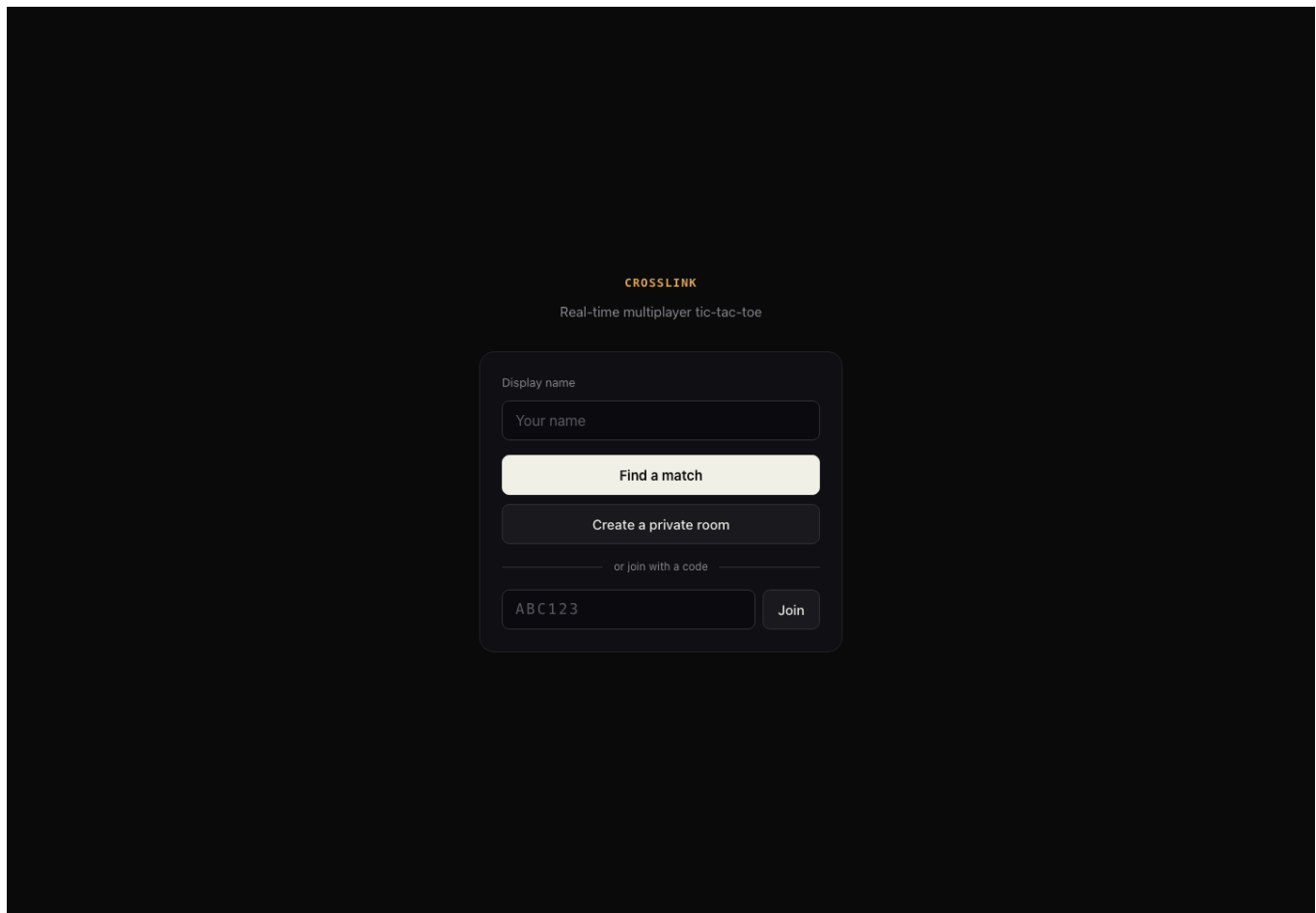


Figure 1. The Crosslink lobby provides quick matchmaking, private-room creation, and room-code joining.

2. System Features

Crosslink implements all seven features required for Project 2.

Project requirement	Crosslink implementation
Turns and instant updates	A player sends a move by HTTP POST. The server applies it and immediately broadcasts the new room state through SSE.
Join and wait for an opponent	A user can find a quick match, create a private room, or join a six-character room code. A room remains in a waiting state until the second player joins.
Same board for both players	Both clients receive snapshots from the same server-owned room. Clients do not decide whether a move is valid.
Player messaging	Each room includes a text chat. Messages are cleaned, limited to 280 characters, stored in order, and sent to both players.
Wins, losses, and draws	The server detects the result and updates both players' statistics after each round. Statistics remain available across rematches during the current server session.
Connection activity	The interface shows recent room creation, joins, moves, messages, results, and rematch events. It also shows whether the live stream is connected.

Project requirement	Crosslink implementation
User-friendly front end	The responsive interface includes clear turn messages, disabled illegal moves, readable room codes, error messages, and keyboard-accessible controls.

Crosslink also supports private invitations, automatic matchmaking, copying a room code, reconnecting the SSE stream, and starting another round only after both players request a rematch.

3. System Analysis and Design

Crosslink uses a client-server architecture. The two browser clients never send data directly to each other. Instead, they communicate with a Next.js server. This design gives the server control over the game and prevents one client from changing the shared board without validation.

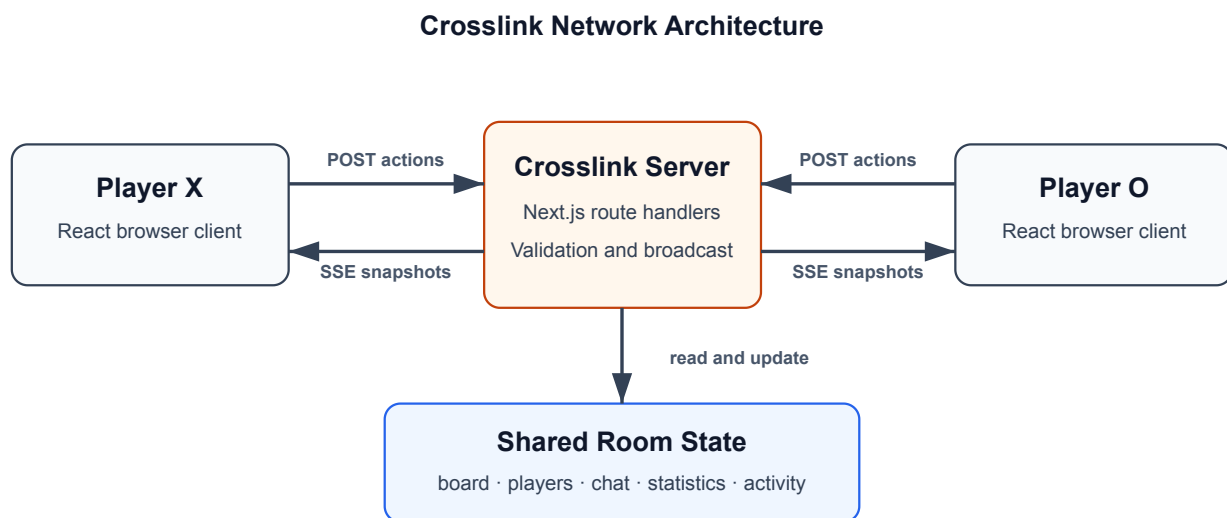


Figure 2. Client actions travel to the server through POST requests, while room snapshots return through SSE.

Each room contains a code, status, board, current turn, winner, round number, two players, rematch votes, messages, and activity records. Rooms are held in one in-memory `Map` keyed by room code. This store is the single source of truth. Browser `sessionStorage` only remembers a temporary room code and player identity so the page can restore a session.

Client actions use normal HTTP POST requests. Separate endpoints handle lobby actions, moves, messages, and rematches. Before accepting a move, the server checks that the room exists, that the player belongs to it, that the round is active, that it is the player's turn, and that the selected square is open. The board changes only after these checks pass.

Each player also opens one long-lived SSE connection. SSE allows an HTTP server to push events to a browser. The server sends a full room snapshot when the connection opens and whenever the room changes. Full snapshots are simple and self-correcting: a reconnecting client receives everything needed to display the correct board. A keepalive comment is sent every 25 seconds so that network intermediaries are less likely to close an idle stream. The browser's `EventSource` object handles automatic reconnection.

The main design limitation is memory storage. If the server stops, all rooms and statistics are lost. This choice reduces setup and keeps the networking logic easy to demonstrate, but it is not suitable for long-term data storage or a multi-server deployment.

4. Implementation Details and Methodology

The application uses Next.js 16, React 19, and TypeScript. Next.js route handlers provide the HTTP API, while React manages the browser interface. Plain CSS supplies the responsive layout without a component library.

The main client component is `app/GameClient.tsx`. It displays either the lobby or the active game. In the lobby, the user enters a display name and chooses quick matching, room creation, or room joining. During a game, the component shows the board, player names and scores, whose turn it is, chat, activity, room code, and live connection status. It sends actions with `fetch` and receives room snapshots with `EventSource`.

The main server module is `lib/game.ts`. It defines the data types and owns the in-memory stores for rooms and SSE listeners. It creates random player IDs with `crypto.randomUUID()`. Room codes contain six characters and avoid visually confusing characters. The module also performs winner detection, matchmaking, message storage, activity logging, snapshot creation, subscription management, and broadcasting. Message history is limited to 60 items, and activity history is limited to 20 items so a room cannot grow without a bound during normal use.

Five API routes divide the server responsibilities:

- `POST /api/lobby` creates or joins rooms and assigns X or O.
- `GET /api/rooms/:code/stream` opens the SSE stream after checking room membership.
- `POST /api/rooms/:code/move` validates a move, updates the board, detects the result, and updates scores.
- `POST /api/rooms/:code/message` validates, stores, and broadcasts a chat message.
- `POST /api/rooms/:code/rematch` records votes and resets the round after both players agree.

During development, polling was considered as a way to update the board. Polling works, but it repeatedly requests the full state even when nothing changes. Crosslink instead uses POST for the smaller client-to-server action channel and SSE for the larger server-to-client update channel. This fits the application's traffic pattern because clients send occasional actions while the server must notify both players. WebSockets would also work, but SSE provides the required one-way push and browser-managed reconnection with less connection code.

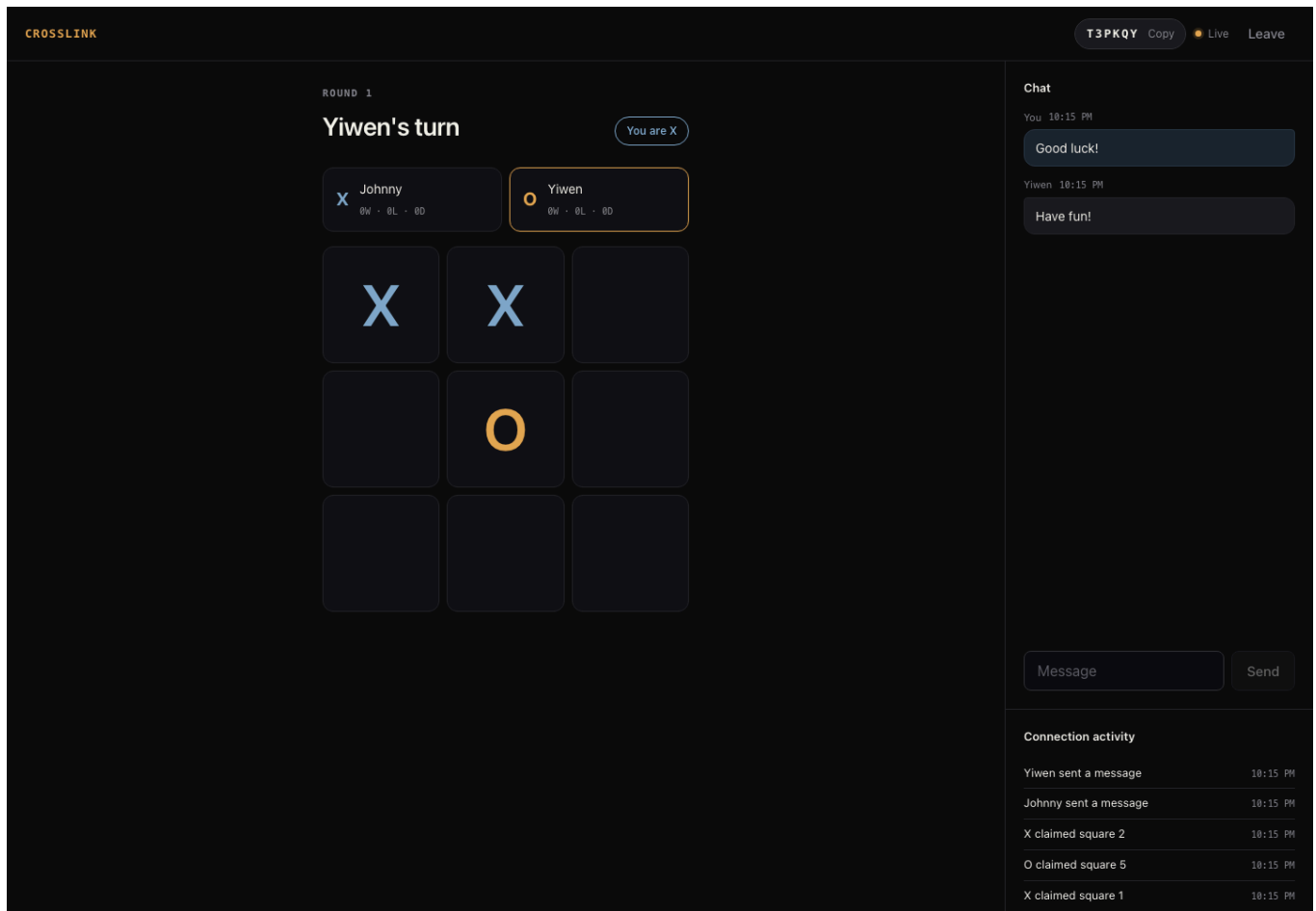


Figure 3. An active room after network actions were applied. The screen shows the shared board, current turn, live connection, chat, and activity log.

5. Testing and User Manual

5.1 Testing

Testing combined static checks, a production build, API checks, and browser inspection. The verification environment used macOS 15.7.7, Node.js 22.14.0, npm 10.9.2, and Google Chrome 150.0.7871.187.

Temporary player pairs were created through the API to test server behavior. The application was also opened in Chrome to confirm that the lobby and active room rendered correctly. Figures 1 and 3 were captured from the production build in this environment.

Test	Expected result	Result
ESLint check	<code>npm run lint</code> finishes without an error	Pass
Production build	TypeScript and Next.js production build complete	Pass
Create and join room	Second player joins and room changes to <code>playing</code>	Pass
Valid move	Server accepts the move and changes the board	Pass
Out-of-turn move	Server rejects it with HTTP 409	Pass
Occupied square	Server rejects it with HTTP 409	Pass
Win detection	Three marks in a row produce winner X or O	Pass
Draw detection	A full board without a winning line produces <code>draw</code>	Pass
Chat	Valid message is stored and returned to the room	Pass
Rematch	New round begins only after both players vote	Pass
Browser state	Board, chat, activity, and live indicator render together	Pass

The checks covered the main success paths and important invalid moves. More automated unit and end-to-end tests would still improve repeatability.

5.2 User Manual

1. Install Node.js 22.13 or newer.
2. Clone the repository and open its directory in a terminal.
3. Run `npm ci` to install the locked dependencies.
4. Run `npm run dev`, then open `http://localhost:3000` in two browser windows.
5. Enter a display name in each window. Choose **Find a match** in both windows, or create a private room in one window and join its code in the other.
6. Player X moves first. Select an empty square when the screen says it is your turn. Moves, chat, scores, and activity appear for both players.
7. After a win or draw, both players can select **Play again** to start the next round.
8. Use **Leave** to return to the lobby. Run `npm run lint` for the static code check and `npm run build` to verify a production build.

Because state is stored in memory, stopping the server removes active rooms. A browser with an old session will return to the lobby when that room can no longer be found.

6. Improvements and Future Work

The first improvement would be persistent storage. A database could save accounts, rooms, messages, and lifetime statistics across restarts. Shared storage or a message broker would also be needed if the application ran on multiple server instances.

Connection handling could be improved by detecting players who leave permanently and deleting abandoned rooms. Authentication could protect saved profiles if accounts are added. Automated tests should cover the winner function, every API validation branch, SSE reconnection, and complete two-browser games. Other possible features include spectator mode, accessibility testing, tournament rooms, and voice chat. WebSockets could replace SSE if future gameplay created frequent two-way traffic, but SSE remains suitable for the current design.

7. Conclusion

Crosslink meets the Project 2 requirements by connecting two players through a real-time, server-authoritative Tic-Tac-Toe game. It supports matchmaking, private rooms, synchronized moves, chat, score tracking, connection activity, and a clear front end. HTTP POST requests keep player actions simple, while SSE pushes the same room state to both browsers. Server-side validation protects turn order and board consistency. The largest limitation is that state is not persistent, but the current design is appropriate for demonstrating networking concepts in a focused class project.

8. Reflection

This project showed us that a simple game can still contain important networking problems. The game rules were easy to write, but coordinating two browsers required clear ownership of the shared state. Making the server authoritative was the most important design decision because it gave both players one reliable board and one place for validation.

We also learned that choosing a transport should depend on the direction and frequency of the data. Polling was easy to understand, but it sent requests when nothing had changed. SSE matched this project better because most updates travel from the server to the players. Working on room codes, reconnecting sessions, invalid moves, and rematch votes also showed us how many edge cases appear when users act at different times.

If we started again, we would plan automated tests and persistence earlier. We would also record network measurements in a repeatable test script instead of measuring them only during development. Overall, the project improved our understanding of HTTP APIs, long-lived connections, server-side validation, shared state, and the tradeoffs between a simple prototype and a production-ready system.